

DeepGuard

Spacecraft AI Safety Monitor: Detecting Problems Before They Become Disasters

Student	Robert B Thompson
Course	ITAI 2372 — AI Applications
Track	Choice 2: Conceptual Design
Institution	Houston Community College
Submission	May 5, 2026
Repository	FP_Robert_B_Thompson_ITAI2372

24 min

One-way signal delay to Mars

87%+

Target rate for catching real problems

< 2 sec

Time from detection to action

6

Spacecraft systems monitored

1. Executive Summary

When a spacecraft is millions of miles from Earth, a radio signal takes up to 24 minutes to reach the ground and another 24 minutes to return. In that window, a problem that starts small, such as a slow temperature rise or a pressure reading drifting out of range, can escalate into a mission-ending failure. The crew cannot wait for instructions, and ground controllers cannot react fast enough.

DeepGuard is a conceptual AI system designed to fill that gap. It watches all spacecraft sensors continuously, spots readings that look wrong before they cross a danger threshold, and takes a pre-approved corrective action on its own, all within two seconds. It does not guess. It only takes actions that mission planners approved before launch, and it logs every decision so engineers can review what happened and why.

2. Problem Statement

2.1 Why Spacecraft Need Onboard AI

Today's spacecraft use fault protection systems built on fixed rules. A rule might say: if temperature exceeds 120 degrees Celsius, sound an alarm. That works well in Earth orbit, where ground control can respond within minutes. It fails in deep space for three reasons.

- **Slow failures slip through.** A sensor drifting toward its limit over 48 hours never triggers a fixed threshold until it is too late.

- **Sensors drift with age.** Over a three-year Mars mission, sensors shift from their original calibration. Thresholds that worked at launch produce constant false alarms by year two.
- **New problems are not in the rulebook.** Fixed rules only catch failures that engineers predicted in advance. Unexpected combinations of events go undetected.

2.2 The Target Missions

DeepGuard is designed for NASA's Artemis lunar Gateway station and future crewed Mars missions. Both involve long stretches where the crew is on their own, communication is delayed, and a single undetected equipment failure could end the mission.

2.3 Project Objectives

- Detect spacecraft equipment problems within 2 seconds using a machine-learning model trained on real NASA sensor data.
- Predict where each sensor reading is heading 15 minutes into the future so the system acts before a problem becomes critical.
- Sort each detected problem into one of four alert levels: Watch, Caution, Warning, or Emergency, and take the matching approved action.
- Produce 40% fewer false alarms than current fixed-threshold systems so the crew is not distracted by non-events.
- Run on radiation-resistant spacecraft hardware using no more than 15 watts of power.
- Log every automated decision with a full record of what triggered it so engineers can audit the system after the mission.

3. Detailed Solution Plan

3.1 How DeepGuard Works

DeepGuard runs three AI tools in sequence. First, it checks whether any current sensor reading looks wrong. Second, it forecasts where that sensor is heading. Third, it decides what to do about it. The whole process takes under two seconds.

Step	Tool Name	Plain-English Job	Output
1	Sensor Watcher	Reads all 6 spacecraft systems 10 times per second and flags any readings that don't fit the normal pattern.	OK or Suspicious flag
2	Trend Forecaster	Looks at the last 8 minutes of data and predicts where each sensor is heading for the next 15 minutes.	Forecast for each sensor
3	Alert Classifier	Combines the suspicious flag and the forecast to decide how serious the situation is.	Caution / Warning / Emergency
4	Action Engine	Picks the correct pre-approved response, carries it out, and writes a full audit record.	Action taken

3.2 Tool 1: The Sensor Watcher

The Sensor Watcher is an AI tool called an Isolation Forest. Think of it this way: normal sensor readings cluster together like a flock of birds flying in formation. An unusual reading sits apart from the flock. The tool finds those isolated readings by randomly slicing the data until it separates each point. Unusual points get isolated quickly. Normal points take many cuts to separate. A reading that gets isolated very fast receives a high suspicion score.

Why this tool? Most AI detection tools need you to first show them many examples of failures. The Sensor Watcher does not. It finds what looks different from normal, even if that type of failure has never happened before. That matters in space, where new failure combinations are always possible.

Setting	Value	What It Means in Plain English
Number of independent checks	200	Uses 200 separate checks in parallel to reduce the chance of a wrong call
Expected fault rate	1%	Based on NASA's published historical fault rate for crewed spacecraft
Alert threshold	Suspicion score below 0.15	At this low means the reading is clearly outside the normal pattern
Sensors checked	All sensors simultaneously	Cross-checks readings across all systems to catch problems that span more than one system
How often it re-learns	Every 30 mission days	Adjusts its idea of normal to account for natural sensor aging over time

3.3 Tool 2: The Trend Forecaster

A reading that is currently fine can still be a problem if it is moving in the wrong direction. The Trend Forecaster reads the last 8 minutes of sensor history and predicts where each reading will be 15 minutes from now. If the prediction shows a sensor heading toward a danger zone, the system acts early instead of waiting for the reading to actually cross the danger line.

This tool uses a type of AI that is designed for sequences of data that change over time, the same kind used in weather forecasting and industrial equipment monitoring. It learns the shape of normal trends and raises an alert when a sensor starts following an abnormal pattern.

Setting	Value	What It Means in Plain English
How far back it looks	Last 8 minutes of data	Enough history to spot a developing trend without carrying unnecessary
How far ahead it predicts	Next 15 minutes	Gives time to act before the problem becomes critical
Layers of pattern learning	2 layers (large, then refined)	First layer finds broad trends; second layer refines the details
Average prediction error target	Less than 2% of each sensor's full range	Small enough that the forecast is reliable for safety decisions
Time to produce each forecast	Under 50 milliseconds	Fast enough to update 10 times per second on spacecraft hardware

3.4 Tool 3: The Alert and Action System

Once the Sensor Watcher flags a problem and the Trend Forecaster confirms where things are heading, the Alert and Action System decides what to do. It uses a decision tree: a series of yes-or-no checks that lead to a specific alert level and a specific response. The four alert levels are:

Alert Level	Suspicion Score	Trend Direction	What DeepGuard Does	How the Crew Finds Out
Watch	Slightly unusual (-0.15 to -0.20)	Stable or improving	Records the event and keeps monitoring	Included in the next routine report
Caution	Moderately unusual (-0.20 to -0.30)	Slowly getting worse	Adjusts system settings to slow the trend	Crew receives a priority alert
Warning	Significantly unusual (-0.30 to -0.45)	Quickly getting worse	Runs the pre-approved fix for this fault type	Immediate crew alert
Emergency	Severely unusual (below -0.45)	Critical level imminent	Starts the emergency shutdown sequence	Emergency alarm + distress beacon

Key Safety Rule: DeepGuard never takes an action that cannot be undone, such as venting gas or sealing off a module, without the crew confirming it first. The only exception is if the crew is incapacitated and the emergency shutdown is the only way to save the spacecraft.

3.5 Spacecraft Systems Covered

Spacecraft System	Plain-English Description	Example Problem DeepGuard Would Catch
Thermal Control	Keeps the spacecraft at safe temperatures	Thermistors slowly drifting hotter than normal over 6 hours
Power Distribution	Manages electricity from solar panels and batteries	Solar panels producing less power than expected after a micrometeorite strike
Life Support	Keeps the air breathable and cabin pressure stable	Oxygen levels rising faster than normal because a filter is degrading
Navigation and Steering	Keeps the spacecraft pointing in the right direction	Inertial navigation sensors giving different readings, suggesting one has drifted
Propulsion	Manages engines and fuel	An unusual fuel pressure reading before a scheduled engine burn
Communications	Handles antennas and data links to Earth	An antenna drifting off its target angle by more than the normal tolerance

3.6 Data Sources

DeepGuard's AI tools are trained and validated using publicly available NASA datasets and peer-reviewed research.

Source	What It Is	How DeepGuard Uses It
NASA Open Data Portal (data.nasa.gov)	Historical sensor logs from the International Space Station	Trains the Sensor Watcher on real examples of normal and faulty spacecraft
NASA SpaceCube-3 Benchmark	Performance data from NASA's radiation-hardened 3U CubeSat	Confirms DeepGuard spacecraft fit under the 15-watt power budget on flight hardware

Curiosity Rover REMS Dataset (pds.nasa.gov)	Environmental readings from Mars Science Laboratory Curiosity rover	Simulates the Curiosity rover temperature swings of a Mars mission for life support
NASA Fault Management Handbook (NASA/TM-2012-217848)	NASA's official guide to classifying and responding to spacecraft faults	Provides high-level alert framework (Watch, Caution, Warning, Emergency)
NASA Lessons Learned Database (llis.nasa.gov)	Records of real spacecraft problems	Supplies near-future scenarios used in worst-case scenario testing

3.7 Hardware Target

DeepGuard runs on the NASA SpaceCube-3, a computer specifically built for space. It is hardened against cosmic radiation, uses only 15 watts (less than a laptop), and delivers enough computing power to run all three AI tools at the same time. It is already scheduled to fly on the Artemis Gateway station. All three DeepGuard tools fit within 2 gigabytes of memory, leaving room for the spacecraft's other software.

3.8 Ethical and Safety Design Choices

- **No irreversible actions without crew approval.** DeepGuard can adjust settings, reduce power loads, and switch to backup systems on its own. It cannot vent gas, seal off a module, or fire thrusters without the crew saying yes.
- **Full audit trail.** Every automated action is recorded with a timestamp, the specific sensor readings that caused it, the system's confidence level, and the action taken. Engineers review this data after every mission.
- **Training data bias addressed.** Most available sensor data comes from Earth-orbit missions. DeepGuard's training set is expanded with simulated Mars-environment data to reduce the risk of errors specific to deep space conditions.
- **Human authority is always final.** The crew can override DeepGuard at any time. The system is a tool, not a commander.

4. Testing Plan

4.1 Testing Approach

DeepGuard follows NASA's four-level testing framework for autonomous spacecraft systems. Testing moves from checking each AI tool individually, to running them all together, to throwing worst-case scenarios at the full system, to having human experts review whether the system's decisions were sensible.

4.2 Level 1: Testing Each Tool Separately

Tool	How It Is Tested	What Must Happen to Pass
Sensor Watcher	500 known problem readings from NASA ISS database are fed into the tool.	The tool must flag problems if flags fewer than 1 in 100.
Trend Forecaster	Uses 20% of the sensor history it has never seen. Average prediction is 15 minutes ahead.	2% of predictions must be correct.
Alert and Action System	200 scripted fault situations are run. The system must choose the correct action.	Must choose the correct action in 200 cases.
Data Input Layer	Corrupted packets and missing sensor signals are fed into the system.	The system keeps running without crashing. Missing values are handled.

4.3 Level 2: Testing All Three Tools Together

- **Response time test.** 10,000 sensor packets are fed into the full pipeline. Every single detected problem must receive an action start within 2 seconds.
- **Multiple problems at once.** Faults are injected in three separate spacecraft systems at the same time. Each must be detected and handled independently.
- **Sensor failure test.** Individual sensors are shut off mid-stream. The system must keep running and not produce false Emergency alerts because a sensor went quiet.
- **Long-run memory test.** The system runs for 72 hours straight in simulation. Memory usage must stay below 2 gigabytes throughout.

4.4 Level 3: Worst-Case Scenario Testing

Fifty fault scenarios drawn from NASA's Lessons Learned database are run ten times each with randomized noise added to the sensor readings. The goal is to confirm the system behaves correctly even when the data is not clean.

Scenario	Type of Fault	System	Expected Alert	Expected Response
S-01	Heater slowly overheating	Thermal	Warning	Switch to backup radiator
S-02	Rapid cabin pressure drop	Life Support	Emergency	Emergency shutdown + alarm
S-03	One solar panel fails	Power	Caution	Reduce power to non-critical systems
S-04	Two nav sensors disagree	Navigation	Watch	Cross-check with star tracker
S-05	Battery voltage falling fast	Power	Warning	Cut power draw by 30%
S-06	CO2 filter losing efficiency	Life Support	Caution	Increase filter cycle rate
S-07	Unusual engine pressure	Propulsion	Watch	Log and keep monitoring

S-08	Antenna drifting off target	Communications	Caution	Switch to backup antenna
------	-----------------------------	----------------	---------	--------------------------

4.5 Level 4: Human Expert Review

Five aerospace engineering students and one faculty advisor review 20 selected scenario decision logs from Level 3. They assess whether each automated action was appropriate given the fault condition, without knowing the intended answer in advance. This validates the system's logic against real human judgment.

To pass all four levels, DeepGuard must: (1) correctly identify more than 87% of real problems, (2) keep false alarms below 5% of all normal events, (3) respond to every Emergency-level fault within 2 seconds, and (4) earn agreement from human reviewers on at least 90% of its autonomous decisions.

4.6 How Success Is Measured

Measurement	What It Checks	Target
Problem detection rate	Out of every 100 real faults, how many does DeepGuard catch?	At least 87 out of 100
False alarm rate	Out of every 100 normal events, how many does it wrongly flag?	More than 5 out of 100
Detection and false alarm balance	A combined score rewarding high detection and low false alarm rate.	Above 0.99
Emergency response time	How fast does the system go from flagging a serious fault to starting the response?	Under 2 seconds
Forecast accuracy	How close are the 15-minute predictions to what the sensor actually reads?	Within sensor's full range
Human agreement	Do aerospace experts agree the system made the right call?	9 of 10 decisions reviewed

5. Project Timeline

Phase	Dates	Work Completed
1 - Research	Apr 6-12, 2026	Reviewed available NASA datasets; confirmed which AI tools fit the problem; finalized the system requirements.
2 - Design	Apr 13-22, 2026	Wrote out full AI tool specifications; built the Alert Level logic tree; defined which spacecraft systems to monitor.
3 - Documentation	Apr 23 - May 1, 2026	Wrote the full project proposal; drafted the testing plan; set up the GitHub repository and README.
4 - Review	May 1-3, 2026	Reviewed and refined the draft; incorporated instructor feedback; completed presentation slides.
5 - Submission	May 4-5, 2026	Final proposal PDF; presentation slides PDF; complete GitHub repository; Canvas submission.

6. References

1. NASA. (2012). *Fault Management Handbook* (NASA/TM-2012-217848). NASA Goddard Space Flight Center.

2. NASA. (2023). *NASA Open Data Portal*. Retrieved from <https://data.nasa.gov>

3. NASA. (2023). *SpaceCube-3: Reconfigurable Space Computing Platform*. NASA GSFC Technology Transfer Office.

4. NASA. (2023). *Mars Science Laboratory REMS Instrument Dataset*. Planetary Data System. <https://pds.nasa.gov>
5. NASA. (2023). *Lessons Learned Information System (LLIS)*. Retrieved from <https://llis.nasa.gov>
6. NASA. (2023). *Artemis Program: Gateway and Deep Space Logistics*. Retrieved from <https://www.nasa.gov/artemis>
7. Liu, F. T., Ting, K. M., and Zhou, Z.-H. (2008). Isolation forest. *2008 Eighth IEEE International Conference on Data Mining*, 413-422.
8. Hochreiter, S., and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735-1780.
9. Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
10. Abadi, M., et al. (2016). TensorFlow: A system for large-scale machine learning. *12th USENIX Symposium on Operating Systems Design and Implementation*, 265-283.